

ESPRIT — POC Vérification IA des dossiers d'admission

Résumé technique d'implémentation

Sprint 1 & Sprint 2

Prétraitement OpenCV → OCR Tesseract → Classification par règles → Extraction structurée

21 juillet 2026

Dépôt : github.com/ab2sfaxii/poc-admission-esprit (branche main)

1. Contexte et objectif

Ce document résume le travail d'implémentation réalisé pour les Sprints 1 et 2 du POC de vérification automatique des dossiers de candidature en admission parallèle (4^e année cycle ingénieur), tel que défini dans le rapport de stage de fin d'études. Le système assiste les enseignants dans la vérification documentaire mais ne décide à aucun moment de l'admission d'un candidat.

Conformément à la planification du rapport (chapitre 6), le Sprint 1 couvre la mise en place de la chaîne prétraitement–OCR, et le Sprint 2 couvre la classification par règles et l'extraction structurée des champs, avec un contrat JSON commun à tous les extracteurs. La vérification de cohérence, le score de complétude et la génération de questions d'entretien (LLM) relèvent du Sprint 3 et ne sont pas couverts par ce travail.

2. Sprint 1 — Prétraitement OpenCV et OCR Tesseract

2.1 Modules livrés

Fichier	Rôle
preprocessing/image_utils.py	Chargement image, niveaux de gris, égalisation d'histogramme, correction gamma, débruitage (filtre médian), recadrage sur le contenu, redimensionnement à une largeur cible, binarisation (seuillage d'Otsu).
preprocessing/deskew.py	Estimation de l'angle d'inclinaison (minAreaRect sur le masque de premier plan) et correction par rotation.
ocr/tesseract_engine.py	Interface avec pytesseract : extraction de texte brut et extraction avec scores de confiance par mot. Langues activées : français, arabe, anglais (fra+ara+eng).
ocr/text_normalization.py	Normalisation du texte OCR (minuscules, suppression des accents) utilisée par la classification et l'extraction pour tolérer les variations de casse/accents.
config/settings.py	Paramètres centralisés : chemins, langues OCR, chemin Tesseract, seuils.

2.2 Choix techniques

- Toutes les opérations de prétraitement sont des transformations algorithmiques déterministes (aucun apprentissage) — deux images similaires produisent systématiquement le même résultat, conformément à la distinction établie dans le rapport (§4.2).
- La correction d'inclinaison est bornée (angle plausible $\leq 15^\circ$) pour éviter de « corriger » une mesure aberrante sur une image quasiment vide.
- Le pipeline complet (preprocess_pipeline) enchaîne : niveaux de gris → contraste → débruitage → skew → recadrage → redimensionnement → binarisation, prêt à être transmis à Tesseract.

2.3 Validation

Aucun scan réel n'étant disponible pour ce POC, 5 documents synthétiques ont été générés (texte rendu en image) pour valider la chaîne de bout en bout. L'OCR restitue le texte avec une fidélité proche de 100 % sur ces documents propres, avec une seule erreur mineure observée (« Algorithmique » lu « Algonthmique »), représentative du bruit OCR réel.

3. Sprint 2 — Classification et extraction structurée

3.1 Classification par règles

classification/rules_classifier.py implémente une classification par mots-clés pondérés pour les 5 types de documents (CIN, acte de naissance, baccalauréat, diplôme de licence, relevé de notes). Chaque type dispose d'une liste de mots-clés normalisés avec un poids ; le type retenu est celui dont le score est le plus élevé, sous réserve d'un seuil de confiance minimal (sinon le document est classé « inconnu »).

3.2 Contrat JSON commun (rapport §3.2.4)

Tous les extracteurs héritent de extraction/base_extractor.py (BaseExtractor) et produisent une sortie strictement identique dans sa structure :

```
{
  "document_type": "cin",
  "fields": {
    "nom": "BEN ALI",
    "prenom": "SALMA",
    "date_naissance": "12/04/2001",
    "numero_document": "08453219"
  },
  "missing_fields": [],
  "warnings": [],
  "extraction_status": "success"
}
```

Le statut est calculé automatiquement : success si tous les champs requis sont présents, partial si certains manquent mais des champs ont été extraits, failed si aucun champ n'a pu être extrait.

3.3 Les 5 extracteurs spécialisés

Type de document	Fichier	Champs requis
cin	extraction/cin_extractor.py	nom, prenom, date_naissance, numero_document
acte_naissance	extraction/acte_naissance_extractor.py	nom, prenom, date_naissance
bac	extraction/bac_extractor.py	nom, prenom, annee_obtention
diplome_licence	extraction/licence_extractor.py	nom, prenom, specialite, annee_obtention
releve_notes	extraction/releve_extractor.py	annee_universitaire, moyenne

L'extracteur de relevé de notes extrait en plus une liste structurée des matières et notes détectées, ainsi que le résultat (validé/non validé) — les lignes d'en-tête connues (année, étudiant, moyenne, résultat) sont écartées avant de rechercher les paires matière/note.

3.4 Modèles de données et orchestration

- models/candidate_file.py : Document, ExtractionResult, CandidateFile (dataclasses), conformes au diagramme de classes du rapport (§3.5).
- main.py : orchestre prétraitement → OCR → classification → extraction pour chaque image de data/samples/, affiche le JSON de chaque document et écrit un rapport agrégé dans data/reports/.

4. Architecture du projet

```
Poc admission/
├── main.py
├── config/
│   ├── __init__.py
│   └── settings.py
├── preprocessing/
│   ├── image_utils.py
│   └── deskew.py
├── ocr/
│   ├── tesseract_engine.py
│   └── text_normalization.py
├── classification/
│   └── rules_classifier.py
├── extraction/
│   ├── base_extractor.py
│   ├── cin_extractor.py
│   ├── acte_naissance_extractor.py
│   ├── bac_extractor.py
│   ├── licence_extractor.py
│   └── releve_extractor.py
├── models/
│   └── candidate_file.py
├── tests/
│   ├── conftest.py
│   ├── sample_documents.py
│   ├── test_preprocessing.py
│   ├── test_ocr.py
│   ├── test_classification.py
│   ├── test_extraction.py
│   └── test_pipeline.py
├── data/
│   ├── samples/      (documents d'exemple générés)
│   └── reports/      (rapports JSON produits)
├── requirements.txt
└── README.md
```

Cette arborescence suit fidèlement le découpage modulaire présenté au chapitre 5.1 du rapport, à l'exception des dossiers `verification/`, `scoring/` et `interview/` (Sprint 3, hors périmètre de ce travail).

5. Stratégie de tests et résultats

5.1 Jeu de documents de test

En l'absence de scans réels au moment de l'implémentation, 5 documents synthétiques représentatifs (CIN, acte de naissance, bac, licence, relevé de notes) ont été générés par rendu de texte en image (`tests/sample_documents.py`), permettant de valider la chaîne complète sans dépendre de données externes. Ils sont destinés à être remplacés par de vrais scans dès qu'ils seront disponibles.

5.2 Résultats

Fichier de test	Ce qui est vérifié	Résultat
<code>test_preprocessing.py</code>	Chargement image, niveaux de gris, redimensionnement, binarisation, correction d'inclinaison, pipeline complet	7/7 OK
<code>test_ocr.py</code>	Présence des mots-clés attendus dans le texte OCR, scores de confiance	2/2 OK

test_classification.py	Classification correcte des 5 types, confiance, cas « inconnu »	8/8 OK
test_extraction.py	Contrat JSON, champs extraits par type, statuts success/partial/failed	10/10 OK
test_pipeline.py	Test fonctionnel de bout en bout sur les 5 documents	5/5 OK

Total : 31 / 31 tests passés.

5.3 Anomalie détectée et corrigée pendant les tests

Le premier test de correction d'inclinaison a révélé une inversion de signe dans preprocessing/deskew.py : la rotation était appliquée dans le même sens que le biais mesuré au lieu de le compenser, doublant l'inclinaison au lieu de la corriger (angle mesuré 10° → 20° après « correction »). Le correctif a consisté à inverser le signe de l'angle transmis à la rotation ; le test est repassé au vert après correction.

6. Gestion de version (Git)

Le dépôt Git initialement présent sur le poste était en réalité racine sur l'ensemble du dossier utilisateur (C:\Users\HP) et connecté au dépôt distant « PIWeb » de l'équipe-projet (github.com/nourboulaaba/PIWeb.git) — un dépôt sans rapport avec ce POC personnel de fin d'études.

Un nouveau dépôt Git indépendant a donc été initialisé directement dans le dossier « Poc admission », avec un commit unique regroupant l'ensemble du travail Sprint 1 & 2 (36 fichiers), puis poussé sur :

```
https://github.com/ab2sfaxii/poc-admission-esprit
Branche : main
```

7. Installation et exécution

Depuis le dossier « Poc admission » (PowerShell) :

```
# Activer l'environnement virtuel
.venv\Scripts\Activate.ps1

# Lancer les tests (31 tests)
pytest -v

# Exécuter le pipeline complet sur data/samples/
python main.py

# Régénérer les documents synthétiques si besoin
python -m tests.sample_documents
```

Tesseract OCR doit être installé sur le poste avec les langues fra, ara, eng (déjà vérifié et disponible). Pour tester avec de vrais scans, il suffit de déposer les images dans data/samples/ puis de relancer python main.py.

8. Périmètre non couvert (Sprint 3) et limites actuelles

- Vérification de cohérence entre documents (identité, années universitaires, doublons, niveau académique) — module verification/ du rapport.
- Calcul du score de complétude et du statut global du dossier (complet / à vérifier / incomplet) — module scoring/.

- Génération automatique de questions d'entretien via un LLM — module interview/.
- Les extracteurs actuels sont basés sur des règles/expressions régulières calées sur le format des documents synthétiques ; ils devront être testés et ajustés sur de vrais scans tunisiens et internationaux (cf. limites discutées au chapitre 8 du rapport).

Document généré automatiquement à partir des travaux réalisés dans la session de développement du 21 juillet 2026.